

CSCE 608 DATABASE SYSTEMS · MODULE 11 · LECTURE NOTES

Logging and Recovery

Surviving the crash, which is the whole point.

MODULE OUTCOMES

- State the write-ahead logging protocol and both of its rules.
- Explain steal and force policies and why no-force/steal is chosen.
- Implement ARIES-style analysis, redo, and undo.
- Explain checkpointing and its trade-off.
- Test recovery honestly.

1 Write-ahead logging

THE TWO RULES

Rule 1, the undo rule: before a modified page may be written to disk, the log record describing that modification must already be on disk.

Rule 2, the redo rule: before a transaction may be reported to the client as committed, all of its log records must be on disk.

Rule 1 provides atomicity. If we crash after a partially completed transaction's pages have reached disk, the log contains the before-images needed to undo them.

Rule 2 provides durability. If we crash after telling the client the transaction committed, but before its pages reached disk, the log contains the after-images needed to redo them.

These two rules are the entire foundation of crash recovery, and everything in this module is machinery for implementing them efficiently.

WHY LOG AT ALL, INSTEAD OF WRITING PAGES CAREFULLY

A single transaction may modify pages scattered anywhere across the database. **Writing those pages is random I/O** — the slow case from Module 02 — and, more fundamentally, **there is no way to make many separate random writes atomic**. A crash partway through leaves some applied and some not, with nothing recording which.

The log is append-only. Writing it is a sequential append, which is the fastest thing storage does, and a single sequential write can be made durable with one fsync.

So: write the log synchronously, and the data pages lazily. The commit path pays one sequential fsync; the scattered page writes happen later, in the background, coalesced and reordered for efficiency.

And batch the fsyncs. Group commit collects the log records of many concurrent transactions and flushes them with a single fsync, so the per-transaction cost of durability falls by an order of magnitude. This is why a database can commit thousands of transactions per second on hardware that performs a few hundred fsyncs per second.

Policy	Definition	Requires	Runtime cost
Steal	The buffer pool may evict and write a page containing uncommitted changes.	UNDO logging	None — the pool evicts freely.
No-steal	Pages with uncommitted changes must stay in memory until commit.	No undo needed.	The buffer pool must hold every uncommitted page , which bounds transaction size by memory.
Force	All of a transaction's pages are written to disk before commit returns.	No redo needed.	Commit performs scattered random writes and is correspondingly slow.
No-force	Pages are written lazily after commit.	REDO logging	None — commit writes only the log.

Every real system chooses steal and no-force, which requires *both* undo and redo — the most complex recovery algorithm of the four combinations. **It is chosen because the alternatives pay at runtime**, continuously, while the complexity is paid once by the implementer and only exercised after a crash. That is a good trade, and recognising it is the point of the table.

2 ARIES

```
Every log record: LSN, txn id, type, prev_LSN
Update records:  + page id, offset, BEFORE image, AFTER image
```

```
Every PAGE stores the LSN of the last record applied to it:
  if page.LSN >= record.LSN, the change is already there: SKIP
  -> this makes REDO IDEMPOTENT
```

```
CLR (Compensation Log Record): written when undoing.
  Records what the undo did; points PAST the record it undid.
  -> undo is never redone, and a crash during undo RESUMES
```

BOTH MECHANISMS EXIST SO THAT CRASHING DURING RECOVERY IS SAFE

Page LSNs make redo idempotent. Each page records the LSN of the last change applied to it, so redo can determine whether a given change is already present and skip it. Applying the redo pass twice produces the same result as applying it once.

Compensation log records make undo idempotent. When undoing a change, ARIES writes a CLR describing the compensating action, and that CLR points *past* the record it undid. So a restarted undo pass skips work already done rather than undoing it twice.

Both exist for one reason: the machine can crash during recovery. This is not a remote possibility — a machine that crashed once because of a failing power supply or a kernel bug is disproportionately likely to crash again during the recovery that follows.

It is also the case nobody tests by accident, which is why §4 requires it explicitly.

- 1 **ANALYSIS.** Scan forward from the most recent checkpoint. Rebuild the **dirty page table** (which pages had unwritten changes) and the **transaction table** (which transactions were active, and what their last LSN was). Determine which transactions were in flight at the crash and must be undone.
- 2 **REDO.** Scan forward from the lowest LSN in the dirty page table, and **repeat history**: reapply every logged change whose page LSN shows it is not already present — **including changes made by transactions that will be undone in the next pass.**
- 3 **UNDO.** Scan backward through the log, rolling back every transaction that was active at the crash, writing a CLR for each change undone, and following prev_LSN pointers to find each transaction's chain of changes.

WHY REPEAT HISTORY, INCLUDING TRANSACTIONS YOU ARE ABOUT TO UNDO

This is the counterintuitive part of ARIES, and the reason is worth stating clearly.

Repeating all of history restores the database to exactly the state it was in at the moment of the crash — whatever that state was, including the partial effects of transactions that never committed.

Undo then operates from a known, well-defined starting point. Without this, undo would have to cope with an unpredictable partial state in which some of a transaction's changes had reached disk and others had not, in an arbitrary combination.

So the algorithm becomes dramatically simpler to implement and to reason about: there is one well-defined intermediate state rather than a combinatorial space of them. **And it is what makes crash-during-recovery work** — a restarted recovery repeats history again, reaches the same intermediate state, and continues from there. Mohan et al., 1992.

3 Checkpoints

CHECKPOINTS TRADE RUNTIME I/O AGAINST RECOVERY TIME

Without checkpoints, recovery would have to scan the log from its beginning — potentially weeks or months of it — because there would be no way to know which changes had already reached disk.

A checkpoint records the dirty page table and the active transaction table, so recovery can begin its analysis there instead.

Frequent checkpoints mean fast recovery and more background I/O during normal operation.

Infrequent checkpoints mean the reverse. The setting is a direct trade between steady-state throughput and the recovery time objective, and it should be chosen from the latter.

Fuzzy checkpoints are what systems actually use. A 'sharp' checkpoint would quiesce the system, flush everything, and record a perfectly consistent state — which means stopping the world, which is unacceptable. A fuzzy checkpoint records the state while transactions continue, accepting that the recorded picture is slightly inconsistent, and leaves the analysis pass to resolve the discrepancy. ARIES is designed around this.

- **Point-in-time recovery.** Restore a backup and replay the log up to any chosen instant. **This recovers from a mistaken UPDATE without a WHERE clause**, not just from a crash — which is the failure mode that actually happens.
- **Replication.** Ship the log to another machine and replay it there. This is how streaming replication works in essentially every system, and it is why a replica is a byte-for-byte copy rather than a logically equivalent one.
- **Change data capture.** Read the log to feed downstream systems — search indexes, caches, analytics — without polling tables or adding triggers. Debezium and similar tools do exactly this.
- **Auditing.** The log is a complete, ordered record of every change ever made.
- **The log turns out to be the most reusable structure in the system**, which is an observation with consequences well beyond databases — it is the central idea of Kafka and of event-sourced architectures generally.

4 Testing recovery

YOU HAVE NOT TESTED RECOVERY UNTIL YOU HAVE KILLED THE PROCESS

Reasoning about recovery establishes nothing. Every system that has ever lost data had a correctness argument, and the argument was wrong in a way nobody had thought of — fsyncgate (Module 02) being the clearest example.

Kill the process at a randomly chosen point during a transactional workload, restart, run recovery, and verify the resulting state against an *independent* model of what should have survived. **Then do it a thousand times**, at a thousand different points.

Use kill -9, not a clean shutdown. A clean shutdown flushes every buffer and runs the shutdown path; it tests nothing about recovery and will pass regardless.

And crash during recovery, repeatedly. That is precisely the case the page LSNs and CLRs of §2 exist to handle, and it is the case that is never exercised by accident.

- **Crash at random points, not convenient ones.** Instrument the code to abort at a randomly selected log write or page flush, so the crash lands in the middle of operations rather than between them.
- **Crash during recovery**, and during the recovery of that recovery.
- **Verify against an independent model** — a separate record of which transactions were acknowledged as committed — rather than against the database's own account of its state. A corrupted database will happily report itself consistent.
- **Report the whole distribution.** A thousand runs and their outcomes, not a single representative success. One failure in a thousand is a durability bug, not noise.
- **Then name a crash point your implementation would not survive, or prove there is none.** Every system rests on assumptions — that fsync means what it says, that a sector write is atomic, that the disk does not reorder across a barrier. Stating yours is what distinguishes an engineer from someone who has not looked. **This is what Project 2 asks for, and the honesty requirement is the point of the exercise.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lectures 20–21, Crash Recovery	WAL, steal/force policies, and the three ARIES passes.
Mohan et al. — ARIES (1992, free)	The original paper. Long, and the repeating-history argument in §2 is worth reading in full.
PostgreSQL documentation — Write-Ahead Logging and Reliability	A production implementation, including group commit and the fsync settings that control durability.

Resource	What it covers for this module
Pillai et al. — All File Systems Are Not Created Equal (free)	What filesystems actually guarantee about ordering and atomicity, and how many applications get it wrong. Sobering, and directly relevant to §4.
Jepsen — database analyses	Empirical crash and partition testing of real systems. The standard §4 describes, applied professionally.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement write-ahead logging with LSNs and before/after images. Verify both WAL rules are enforced by instrumenting every page write.
- 2 Implement group commit and measure the throughput improvement against one fsync per transaction.
- 3 Implement page LSNs and demonstrate redo idempotence: run the redo pass twice and confirm an identical result.
- 4 Implement ARIES analysis, redo, and undo with CLRs.
- 5 **Build the crash test harness:** kill the process at a randomly chosen log write, recover, and verify against an independent record of acknowledged commits.
- 6 Run it a thousand times and report every outcome.
- 7 Crash during recovery and confirm the restarted recovery completes correctly.
- 8 Implement fuzzy checkpointing. Measure recovery time against checkpoint interval and plot it.
- 9 Implement point-in-time recovery: restore a backup and replay to a chosen LSN.
- 10 **Write the failure analysis:** name a crash point your implementation would not survive, and the assumption it rests on.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State both WAL rules and say which ACID property each provides.
- 2 Why log at all, rather than writing data pages carefully?
- 3 Define steal and force, and say which combination everyone chooses and why.
- 4 What makes redo idempotent, and what makes undo idempotent?
- 5 Why do both of those mechanisms exist?
- 6 Name the three ARIES passes and what each accomplishes.

- 7 Why does redo repeat the history of transactions that will be undone?
- 8 What do checkpoints trade, and what is a fuzzy checkpoint?
- 9 Name four things the log enables besides crash recovery.
- 10 What does an honest recovery test require?